JavaScript Interview Question Answer

1. What is JavaScript?

JavaScript is a **scripting language** used to create **dynamic and interactive web content**. It runs in the browser and allows manipulation of HTML, CSS, and APIs.

JavaScript is a high level, object-oriented, lightweight, cross-platform programming language. It is also called a scripting language. A scripting language is a type of programming language.

JavaScript is **single-threaded** — it executes code on a **single main thread**, meaning it can do **only one thing at a time** in the main execution flow.

2. What are the data types in JavaScript?

- **Primitive types:** String, Number, Boolean, Undefined, Null, Symbol, BigInt
- **Non-primitive:** Object (including Array, Function)

3. What is the difference between var, let, and const?

Keyword	Scope	Re-declaration	Re-assignment	Hoisted
var	Function	Yes	Yes	Yes
let	Block	No	Yes	No
const	Block	No	No	No

4. What is == vs ===?

- == → compares **values only** (performs type conversion)
- === → compares **values and data types** (strict equality)

Example:

5 == "5" // true

5 === "5" // false

BitKhanan IT Education

5. What is NaN?

Answer:
NaN means “Not-a-Number”.
Example:

```
parseInt("abc"); // NaN
```

It’s still a **number type** but represents an invalid numeric value.

6. What is typeof operator used for?

It returns the **data type** of a variable.

```
typeof 10; // "number"

typeof "Hi"; // "string"
```

7. What are truthy and falsy values?

Falsy values → false, 0, "", null, undefined, NaN

Truthy → everything else.

8. What is the difference between null and undefined?

- undefined: variable declared but not assigned a value
- null: variable explicitly assigned "no value"

9. What is a function in JavaScript?

A **function** is a block of reusable code.

```
function greet(name) {
  return "Hello " + name;
}
```

10. What are arrow functions?

Short syntax for writing functions:

```
const add = (a, b) => a + b;
```

BitKhanan IT Education

They **don't have their own this** context.

11. What is the difference between call(), apply(), and bind()?

Answer:

```
const person = { name: "Amit" };

function greet(msg) { console.log(msg + " " + this.name); }

greet.call(person, "Hello");  // Hello Amit

greet.apply(person, ["Hi"]);  // Hi Amit

const fn = greet.bind(person, "Hey");

fn(); // Hey Amit
```

12. What is a closure?

A **closure** is a function that remembers variables from its **outer scope** even after the outer function has finished executing.

```
function outer() {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  }
}
const counter = outer();
counter(); // 1
counter(); // 2
```

13. What is the difference between synchronous and asynchronous code?

- **Synchronous:** Executes line by line (blocking)
- **Asynchronous:** Executes without waiting for other operations (non-blocking)

BitKhanan IT Education

Example:

```
console.log("1");
setTimeout(() => console.log("2"), 1000);
console.log("3");
// Output: 1, 3, 2
```

14. What is a Promise?

A Promise in JavaScript is an **object** that provides a way to **handle asynchronous operations** in a more structured and manageable manner.

Promises have three **states**:

Pending:

Fulfilled (Resolved):

Rejected:

Promises have two main **methods** for handling the results of asynchronous operations:

- **then():**
- **catch()**:
- finally():

resolve, reject is a callback function.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Done!"), 1000);
});
p.then(result => console.log(result));
```

15. What are template literals?

Strings with embedded expressions using backticks:

```
let name = "Amit";
```

BitKhanan IT Education

console.log(`Hello ${name}!`);

16. What is destructuring?

Extracting values from arrays or objects:

const [a, b] = [1, 2];

const {name, age} = {name: "Amit", age: 25};

17. What are default parameters?

function greet(name = "Guest") {

  return "Hello " + name;

}

18. What are array methods in JavaScript?

Method	Description / Work	Syntax	Return Value
push()	Adds one or more elements to the end of the array	array.push(element1, element2, ...)	New length of the array
pop()	Removes the last element from the array	array.pop()	Removed element
shift()	Removes the first element from the array	array.shift()	Removed element
unshift()	Adds one or more elements to the beginning of the array	array.unshift(element1, element2, ...)	New length of the array
concat()	Joins two or more arrays	array1.concat(array2, array3, ...)	New array

BitKhanan IT Education

Method	Description / Work	Syntax	Return Value
join()	Joins all elements into a string	array.join(separator)	String
slice()	Returns a shallow copy of part of the array	array.slice(start, end)	New array
splice()	Adds/removes elements at a specific index	array.splice(start, deleteCount, item1, item2, ...)	Array of removed elements
indexOf()	Returns first index of a value	array.indexOf(value, start)	Index or -1
lastIndexOf()	Returns last index of a value	array.lastIndexOf(value, start)	Index or -1
includes()	Checks if a value exists in the array	array.includes(value, start)	true or false
reverse()	Reverses array order (mutates original)	array.reverse()	Reversed array
sort()	Sorts array elements (mutates original)	array.sort(compareFn)	Sorted array
forEach()	Executes a function for each element	array.forEach(callback)	undefined
map()	Creates a new array by applying a function	array.map(callback)	New array
filter()	Creates a new array with elements that pass a condition	array.filter(callback)	New array
reduce()	Reduces array to a single value (accumulator)	array.reduce(callback, initialValue)	Single value
reduceRight()	Same as reduce() but from right to left	array.reduceRight(callback, initialValue)	Single value

BitKhanan IT Education

Method	Description / Work	Syntax	Return Value
find()	Returns first element that satisfies condition	array.find(callback)	Element or undefined
findIndex()	Returns index of first element satisfying condition	array.findIndex(callback)	Index or -1
every()	Checks if all elements pass a condition	array.every(callback)	true or false
some()	Checks if at least one element passes a condition	array.some(callback)	true or false
flat()	Flattens nested arrays	array.flat(depth)	New flattened array
flatMap()	Map + Flat in one step	array.flatMap(callback)	New flattened array
fill()	Fills array elements with a static value	array.fill(value, start, end)	Modified array
copyWithin()	Copies array elements to another position	array.copyWithin(target, start, end)	Modified array
toString()	Converts array to string	array.toString()	String
toLocaleString()	Converts array to localized string	array.toLocaleString()	Localized string
keys()	Returns array iterator with keys (indexes)	array.keys()	Iterator object
values()	Returns array iterator with values	array.values()	Iterator object
entries()	Returns array iterator with key/value pairs	array.entries()	Iterator object

BitKhanan IT Education

Method	Description / Work	Syntax	Return Value
from()	Creates an array from array-like or iterable object	Array.from(object, mapFn)	New array
of()	Creates an array from arguments	Array.of(element1, element2, ...)	New array
isArray()	Checks if a value is an array	Array.isArray(value)	true or false
at()	Returns element at a given index (supports negative index)	array.at(index)	Element or undefined
findLast()	Returns the last element satisfying a condition *(ES2023)*	array.findLast(callback)	Element or undefined
findLastIndex()	Returns the last index satisfying a condition *(ES2023)*	array.findLastIndex(callback)	Index or -1
with()	Returns a copy of array with one element replaced *(ES2023)*	array.with(index, value)	New array

BitKhanan IT Education

19. What is the event loop?

It handles asynchronous operations in JavaScript.
It continuously checks the **call stack** and **callback queue**, executing queued callbacks when the stack is empty.

20. What is DOM?

Document Object Model (DOM) is the programming interface for HTML documents — allows JS to manipulate elements dynamically.

Example:

```
document.getElementById("title").textContent = "Hello JS!";
```

DOM Methods:

Method	What It Does	Syntax Example	Return Value
getElementById()	Selects an element by ID	document.getElementById("myDiv")	Single element or null
querySelector()	Selects the **first** element matching a CSS selector	document.querySelector(".btn")	Single element or null
querySelectorAll()	Selects **all** elements matching a CSS selector	document.querySelectorAll("p")	NodeList (like array)
createElement()	Creates a new HTML element	document.createElement("div")	New element node
appendChild()	Appends a child to a parent element	parent.appendChild(child)	Added child element
append()	Adds element(s) or text at the end	parent.append(newDiv)	undefined
remove()	Removes the selected element	element.remove()	undefined
setAttribute()	Sets or updates an attribute	img.setAttribute("src", "photo.jpg")	undefined
getAttribute()	Gets the value of an attribute	img.getAttribute("src")	String or null
classList.add()	Adds one or more CSS classes	div.classList.add("active")	undefined

BitKhanan IT Education

Method	What It Does	Syntax Example	Return Value
classList.remove()	Removes one or more CSS classes	div.classList.remove("hidden")	undefined
classList.toggle()	Adds/removes a class (toggles it)	div.classList.toggle("dark-mode")	true or false
addEventListener()	Adds an event listener to an element	btn.addEventListener("click", handleClick)	undefined
innerHTML	Gets or sets the HTML inside an element	div.innerHTML = " Hello "	String (HTML content)
textContent	Gets or sets the text inside an element	div.textContent = "Hello!"	String (plain text)

21. Reverse a string in JavaScript

```
let str = "hello";

console.log(str.split("").reverse().join("")); // "olleh"
```

22. Find the maximum number in an array

```
let arr = [10, 20, 30];

console.log(Math.max(...arr)); // 30
```

23. Check if a number is even or odd

```
let num = 5;

console.log(num % 2 === 0 ? "Even" : "Odd");
```

BitKhanan IT Education

24. Swap two variables without a third variable

```
let a = 5, b = 10;

a = a + b;

b = a - b;

a = a - b;
```

25. What is hoisting?

JavaScript moves **variable and function declarations** to the top of their scope before code execution.

```
console.log(a); // undefined

var a = 5;
```

26. What is the difference between deep copy and shallow copy?

- **Shallow copy:** Copies only the first level of the object; nested objects share references.
- **Deep copy:** Copies all levels — nested objects are cloned as well.
  Example:

  ```
  let obj1 = { a: 1, b: { c: 2 } };

  let shallow = { ...obj1 };

  let deep = structuredClone(obj1);

  obj1.b.c = 100;

  console.log(shallow.b.c); // 100 (same reference)

  console.log(deep.b.c); // 2 (different reference)
  ```

27. What are higher-order functions?

Functions that take other functions as arguments or return functions.
Example:

```
function higherOrder(fn) {

  return fn();
```

BitKhanan IT Education

```
}

higherOrder(() => console.log("Hello!"));
```

28. What is the spread and rest operator?

- **Spread (...)** expands array/object values.
- **Rest (...)** collects multiple arguments into an array.
  Example:

```
const arr = [1,2,3];

console.log(...arr); // 1 2 3


function sum(...nums) {

  return nums.reduce((a,b) => a+b);

}

console.log(sum(1,2,3)); // 6
```

29. What are callbacks in JavaScript?

A callback is a function passed as an argument to another function and executed after some operation.
Example:

```
function greet(name, callback) {

  console.log("Hello " + name);

  callback();

}

greet("Amit", () => console.log("Welcome!"));
```

30. What is async/await?

Simplified syntax for handling promises.

```
async function fetchData() {

  const res = await fetch('https://api.example.com');
```

BitKhanan IT Education

```
  const data = await res.json();
  console.log(data);
}
```

✅ Makes asynchronous code look synchronous.

31. What is event bubbling and capturing?

- **Bubbling:** Event propagates from the target element **upward** to the root.
- **Capturing:** Event propagates from root **downward** to the target.
  Example:

```
element.addEventListener("click", handler, true); // capturing
element.addEventListener("click", handler, false); // bubbling
```

32. What is debouncing and throttling?

Both control how often a function executes.

- **Debounce:** Runs after a pause in events.
- **Throttle:** Runs at regular intervals during events.
  Example (debounce):

```
function debounce(fn, delay){
  let timer;
  return function(...args){
   clearTimeout(timer);
   timer = setTimeout(()=>fn(...args), delay);
  }
}
```

BitKhanan IT Education

33. What is the difference between localStorage, sessionStorage, and cookies?

Feature	localStorage	sessionStorage	cookies
Storage limit	~5MB	~5MB	4KB
Expiry	Never (manual clear)	On tab close	Based on expiry
Accessible from	Client only	Client only	Client + Server
Used for	Long-term data	Temporary session	Auth/session data

34. What is a prototype in JavaScript?

Every JS object has a prototype — an object it inherits properties from.
Example:

```
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function() {
  console.log("Hello " + this.name);
}
const p1 = new Person("Amit");
p1.greet(); // Hello Amit
```

35. What is a class in JavaScript?

A class is a blueprint for creating objects.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(` Hello ${this.name}` );
```

BitKhanan IT Education

```
  }
}
const p = new Person("Amit");
p.greet();
```

36. What are modules in JavaScript?

Used to split code into separate files.

```
// math.js
export const add = (a,b) => a+b;

// main.js
import { add } from './math.js';
console.log(add(2,3));
```

37. What are generators?

Special functions that can pause and resume execution using yield.

```
function* gen() {
  yield 1;
  yield 2;
}
const g = gen();
console.log(g.next().value); // 1
console.log(g.next().value); // 2
```

38. What is the difference between map() and forEach()?

Feature	map()	forEach()
Returns new array	✅ Yes	❌ No

BitKhanan IT Education

Feature	map()	forEach()
Chainable	✅ Yes	❌ No
Used for	Transform data	Iterate only

39. What is a pure function?

A function that:

- Always returns the same output for the same input.
- Has no side effects.

function add(a,b){ return a+b; } // pure

40. What is immutability?

Data that cannot be changed after creation.
Example:

const arr = [1,2,3];

const newArr = [...arr, 4]; // does not modify original

41. What is the use of JSON in JavaScript?

Used to store and transfer data between client and server.

const obj = { name: "Amit" };

const str = JSON.stringify(obj); // object → string

const parsed = JSON.parse(str); // string → object

42. What is the difference between for...in and for...of?

Loop	Works On	Iterates Over
for...in	Objects	Keys
for...of	Arrays, strings	Values

Bitkhanan IT Education

43. What is destructuring with default value?

```
const { name="Guest", age } = { age:25 };
console.log(name); // Guest
```

44. What is an Immediately Invoked Function Expression (IIFE)?

A function that runs immediately after its creation.

```
(function(){
  console.log("Runs immediately!");
})();
```

45. What is memoization?

Caching the result of expensive function calls.

```
function memoize(fn){
  const cache = {};
  return function(n){
    if(n in cache) return cache[n];
    cache[n] = fn(n);
    return cache[n];
  }
}
```

46. What are WeakMap and WeakSet?

- **WeakMap:** Keys are objects, garbage collected automatically.
- **WeakSet:** Stores objects only, not primitive values.

```
let obj = {};
let wm = new WeakMap();
wm.set(obj, "info");
```

BitKhanan IT Education

47. What is the Temporal Dead Zone (TDZ)?

The time between block start and variable initialization (for let/const).
Accessing before initialization causes a ReferenceError.

```
console.log(a); // ReferenceError

let a = 5;
```

48. What is the difference between shallow equality (==) and deep equality?

- **Shallow equality:** compares references
- **Deep equality:** compares nested values.

  ```
  let a = {x:1}, b = {x:1};

  console.log(a == b); // false
  ```

49. How to clone an object?

```
const clone1 = {...obj};

const clone2 = Object.assign({}, obj);

const clone3 = structuredClone(obj); // deep copy
```

50. What is the difference between eval() and Function()?

- eval() executes code in current scope.
- Function() creates a new function scope.
  ⚠️ Avoid using both for security reasons.

51. What is the prototype chain?

Every JavaScript object inherits properties and methods from its prototype.
If a property is not found on the object, JS looks up the chain until it reaches Object.prototype.
Example:

```
let arr = [];

console.log(arr.__proto__ === Array.prototype); // true
```

BitKhanan IT Education

console.log(Array.prototype.__proto__ === Object.prototype); // true

52. What is currying in JavaScript?

Currying transforms a function with multiple arguments into a sequence of functions each taking one argument.

```
function add(a) {
 return function(b) {
  return function(c) {
   return a + b + c;
  };
 };
}
console.log(add(1)(2)(3)); // 6
```

53. What is event delegation?

Instead of attaching event listeners to multiple child elements, we attach one listener to a parent and use event bubbling.

```
document.getElementById("list").addEventListener("click", (e) => {
 if(e.target.tagName === "LI") console.log(e.target.textContent);
});
```

✅ Improves performance and simplifies dynamic element handling.

54. What are microtasks and macrotasks?

JavaScript event loop handles:

- **Macrotasks:** setTimeout, setInterval, I/O, events
- **Microtasks:** Promises, process.nextTick, MutationObserver
  Example:

  ```
  setTimeout(() => console.log("Macrotask"));
  Promise.resolve().then(() => console.log("Microtask"));
  ```

BitKhanan IT Education

```
// Output: Microtask → Macrotask
```

55. What is the difference between shallow and deep comparison in objects?

Shallow comparison checks references, deep comparison checks nested values.

```
JSON.stringify(obj1) === JSON.stringify(obj2); // deep comparison (simple way)
```

56. What is the 'this' keyword in JavaScript?

this refers to the current context.

- In a function → global object (window)
- In strict mode → undefined
- In a method → the object calling it
- In arrow function → inherited from outer scope
  Example:

```
const obj = {
  name: "Amit",
  show: function() { console.log(this.name); }
};
obj.show(); // Amit
```

57. What are pure and impure functions?

- **Pure:** No side effects, same output for same input.
- **Impure:** Changes external state or depends on it.
  Example:

```
function pure(a,b){ return a+b; }    // pure
function impure(a){ return a + Math.random(); } // impure
```

58. What is the difference between synchronous and asynchronous iteration?

- **Synchronous:** Works on normal iterables (for…of)
- **Asynchronous:** Works on async data streams (for await…of)

BitKhanan IT Education

59. What are symbols in JavaScript?

A Symbol is a unique and immutable primitive used as object keys.

```
const sym = Symbol("id");
const obj = { [sym]: 123 };
```

60. What are iterators and generators?

- **Iterator:** Object with next() method returning {value, done}.
- **Generator:** Function that automatically creates an iterator using yield.

```
function* gen() {
  yield 1;
  yield 2;
}
const it = gen();
console.log(it.next()); // {value:1, done:false}
```

61. What is a promise chain?

Chaining .then() to handle multiple asynchronous operations sequentially.

```
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

62. What are static methods in classes?

Methods called directly on the class, not on instances.

```
class MathUtil {
  static add(a,b) { return a+b; }
}
```

BitKhanan IT Education

```
console.log(MathUtil.add(2,3)); // 5
```

63. What is destructuring in function parameters?

You can destructure directly inside parameters:

```
function show({name, age}) {
  console.log(name, age);
}
show({name:"Amit", age:25});
```

64. What is optional chaining (?.)?

Prevents errors when accessing nested properties.

```
const user = { profile: { name: "Amit" } };
console.log(user.profile?.name); // Amit
console.log(user.address?.city); // undefined (no error)
```

65. What is nullish coalescing operator (??)?

Returns the right-hand value only if left-hand is null or undefined.

```
let name = null;
console.log(name ?? "Guest"); // Guest
```

66. What is object destructuring with rest properties?

```
const {a, ...rest} = {a:1, b:2, c:3};
console.log(rest); // {b:2, c:3}
```

67. What are tagged template literals?

Custom string processing using functions.

```
function tag(strings, name) {
  return `${strings[0]}${name.toUpperCase()}` ;
```

BitKhanan IT Education

```
}
console.log(tag`Hello ${"amit"}`); // Hello AMIT
```

68. What is the difference between Object.freeze() and Object.seal()?

Method	Can Add Props	Can Modify Props	Can Delete Props
Object.freeze()	❌	❌	❌
Object.seal()	❌	✅	❌

69. What is function hoisting vs variable hoisting?

- **Function declarations** are hoisted entirely.
- **Variables (var)** are hoisted but initialized with undefined.

```
sayHi(); // Works
function sayHi(){ console.log("Hi"); }

console.log(x); // undefined
var x = 10;
```

70. What is dynamic typing in JavaScript?

Variables can hold any type at runtime.

```
let a = 10;
a = "Hello"; // valid
```

71. What are async iterators?

Used to iterate over asynchronous data sources.

```
for await (const item of asyncGenerator()) {
  console.log(item);
}
```

BitKhanan IT Education

72. What is deep cloning using JSON?

```
let clone = JSON.parse(JSON.stringify(obj));
```

⚠️ Note: fails for functions, undefined, and symbols.

73. What are modules — import/export default?

```
// file.js
export default function greet(){ console.log("Hi"); }
// main.js
import greet from './file.js';
```

74. What is function composition?

Combining multiple functions to produce a new one.

```
const compose = (f,g) => (x) => f(g(x));
const add = x => x + 2;
const mult = x => x * 3;
console.log(compose(mult, add)(2)); // (2+2)*3 = 12
```

75. What are private fields in classes?

Declared with # — not accessible outside the class.

```
class User {
  #password = "secret";
  getPassword() { return this.#password; }
}
```

76. What is the difference between shallow and deep copy of arrays?

```
let arr = [1, [2,3]];
let shallow = [...arr];
```

BitKhanan IT Education

```
let deep = structuredClone(arr);

arr[1][0] = 100;

console.log(shallow[1][0]); // 100 (same reference)

console.log(deep[1][0]); // 2
```

77. What is event propagation?

The process by which an event travels through the DOM tree —
Capturing → Target → Bubbling.

78. What is function chaining?

Calling multiple methods sequentially on the same object.

```
class Calculator {
  constructor(val=0){ this.val = val; }
  add(n){ this.val += n; return this; }
  sub(n){ this.val -= n; return this; }
  result(){ return this.val; }
}
console.log(new Calculator(5).add(10).sub(3).result()); // 12
```

79. What are bitwise operators in JavaScript?

Operate on binary representations.

Operator	Description
&	AND
|	OR
^	XOR
~	NOT
<<	Left shift

Bitkhanan IT Education

Operator	Description
>>	Right shift

80. What are WeakRefs (Weak References)?

They let you hold a weak reference to an object without preventing garbage collection.

```
let obj = { name: "Amit" };

let weak = new WeakRef(obj);
```

81. Explain the differences between var, let, and const.

Keyword	Scope	Re-declaration	Re-assignment	Hoisting Behavior
var	Function-scoped	Yes	Yes	Hoisted to the top of its function/global scope and initialized with undefined.
let	**Block-scoped**	No	Yes	Hoisted to the top of its block, but **not initialized**. Accessing it before declaration causes a **Temporal Dead Zone (TDZ)** error.
const	**Block-scoped**	No	No	Hoisted to the top of its block, but **not initialized**. Must be initialized upon declaration.

82. What would be the result of 3+2+"7"+ 5 +1?

5751

83. Is JavaScript compiled or interpreted?

JavaScript is mostly interpreted, but modern browsers also compile it just-in-time (JIT) to make it faster.

- The browser reads your JavaScript code line by line and runs it directly.

BitKhanan IT Education

- Modern browsers (like Chrome's V8 engine) first translate parts of your code into machine code while running it, so the computer can execute it much faster.

84. What's the return-value difference between x++ and ++x?

Both increment, but return different values.

- x++: **post-increment** → returns the old value, then increments.
- ++x: **pre-increment** → increments first, then returns the new value.

85. What is a Variable Scope in JavaScript?

In JavaScript, we have each variable are accessed and modified through either one of the given scope:

- **Global Scope**: Outermost level (accessible everywhere).
- **Local Scope**: Inner functions can access variables from their parent functions due to lexical scoping.
- **Function Scope**: Variables are confined to the function they are declared in.
- **Block Scope**: Variables declared with let or const are confined to the nearest block (loops, conditionals, etc.).

86. What does this code log?

```
const arr = [1, 2, 3];

arr[10] = 99;

console.log(arr.length);
```

11

87. What is negative infinity?

The negative infinity is a constant value represents the lowest available value. It means that no other number is lesser than this value. It can be generate using a self-made function or by an arithmetic operation. JavaScript shows the NEGATIVE_INFINITY value as -Infinity.

BitKhanan IT Education

88. What are "truthy" and "falsy" values in JavaScript

- **Falsy**: false, 0, "" (empty string), null, undefined, NaN.
- **Truthy**: Everything else (e.g., any non-empty string, any non-zero number, objects, arrays).

89. What are undeclared and undefined variables?

- Undefined**:** It occurs when a variable is declare but not assign any value. Undefined is not a keyword.
- Undeclared**:** It occurs when we try to access any variable which is not initialize or declare earlier using the var or const keyword. If we use 'typeof' operator to get the value of an undeclare variable, we will face the runtime error with the return value as "undefined". The scope of the undeclare variables is always global.

90. What will be the result of this expression?

```
console.log(null ?? 'default');

console.log(undefined ?? 'default');

console.log(false ?? 'default');
```

default

default

false

91. What will be the output of this code?

```
let x = 0;

console.log(x++);

console.log(++x);
```

0

2

BitKhanan IT Education

92. What is the output of this snippet?

```
const a = [1, 2, 3];

const b = [1, 2, 3];

console.log(a == b, a === b);
```

false

false

Explanation: Arrays are objects and compared by reference. a and b are distinct objects, so both loose (==) and strict (===) comparisons yield false.

93. Is JavaScript statically typed or dynamically typed?

JavaScript is dynamically typed because we don't have to tell JavaScript what kind of data (number, text, true/false, etc.) a variable will hold when you create it. The type is decided automatically when the program runs.

94. Is JavaScript single-threaded or multi-threaded

JavaScript is single-threaded, but it uses asynchronous features to handle multiple tasks efficiently.

- JavaScript runs on a single main thread, meaning it executes one command at a time in a sequence.
- It was designed this way to simplify programming for the browser and avoid issues like race conditions when accessing the DOM.

95. What are the types of Pop up boxes available in JavaScript?

There are three types of pop boxes available in JavaScript.

- Alert
- Confirm
- Prompt

BitKhanan IT Education

96. Explain the concept of "hoisting" in JavaScript

Hoisting is a JavaScript mechanism where variable and function declarations are moved ("hoisted") to the top of their containing scope during the compile phase.

- **Variable declarations (var)**: Declarations are hoisted, but not initializations. The value of the variable is undefined if accessed before initialization.
- **Variable declarations (let and const)**: Declarations are hoisted, but not initialized. Accessing them results in ReferenceError until the actual declaration is encountered.
- **Function expressions (var)**: Declarations are hoisted, but not initializations. The value of the variable is undefined if accessed before initialization.
- **Function declarations (function)**: Both declaration and definition are fully hoisted.
- **Class declarations (class)**: Declarations are hoisted, but not initialized. Accessing them results in ReferenceError until the actual declaration is encountered.
- **Import declarations (import)**: Declarations are hoisted, and side effects of importing the module are executed before the rest of the code.

The following behavior summarizes the result of accessing the variables before they are declared.

Declaration	Accessing before declaration
var foo	undefined
let foo	ReferenceError
const foo	ReferenceError
class Foo	ReferenceError
var foo = function() { ... }	undefined
function foo() { ... }	Normal
import	Normal

on IT Education

97. Describe the difference between a cookie, sessionStorage and localStorage in browsers

Property	Cookie	localStorage	sessionStorage
Initiator	Client or server. Server can use Set-Cookie header	Client	Client
Lifespan	As specified	Until deleted	Until tab is closed
Persistent across browser sessions	If a future expiry date is set	Yes	No
Sent to server with every HTTP request	Yes, sent via Cookie header	No	No
Total capacity (per domain)	4kb	5MB	5MB
Access	Across windows/tabs	Across windows/tabs	Same tab
Security	JavaScript cannot access HttpOnly cookies	None	None

98. Describe the difference between <script>, <script async> and <script defer>

Feature	<script>	<script async>	<script defer>
Parsing behavior	Blocks HTML parsing	Runs parallel to parsing	Runs parallel to parsing

Feature	<script>	<script async>	<script defer>
Execution order	In order of appearance	Not guaranteed	In order of appearance
DOM dependency	No	No	Yes (waits for DOM)

99. What's the difference between a JavaScript variable that is: null, undefined or undeclared?

Trait	null	undefined	Undeclared
Meaning	Explicitly set by the developer to indicate that a variable has no value	Variable has been declared but not assigned a value	Variable has not been declared at all
Type (via typeof operator)	'object'	'undefined'	'undefined'
Equality Comparison	null == undefined is true	undefined == null is true	Throws a ReferenceError

100. What are the differences between Map/Set and WeakMap/WeakSet in JavaScript?

Feature	Map	WeakMap	Set	WeakSet
Key Types	Any data type	Objects (weak references)	Any data type (unique)	Objects (weak references, unique)
Garbage Collection	Keys and values are	Keys can be garbage collected if not	Elements are not garbage collected	Elements can be garbage collected if not

Feature	Map	WeakMap	Set	WeakSet
	not garbage collected	referenced elsewhere		referenced elsewhere
Use Cases	General-purpose key-value storage	Caching, private DOM node data	Removing duplicates, membership checks	Object weak references, custom use cases

BitKhanan IT Education